

Święta Księga C++

Tom II: Amunicja (Karta.h)

Ten plik nie dziedziczy po niczym. Jest to prosty kontener na dane, tzw. struktura danych z akcjami.

1. Typy Wyliczeniowe (w uproszczeniu)

```
int typJednostki; // 1 = Miecznik, 2 = Lucznik, 3 = Mag
```

Zamiast pisać trudne w porównywaniu teksty (Stringi), używamy zwykłych liczb całkowitych (Integerów) do oznaczania typów kart. Z punktu widzenia kompilatora łatwiej i szybciej jest sprawdzić warunek `if (typ == 1)` niż `if (typ == "Miecznik")`. W mechatronice to odpowiednik przypisania konkretnych stanów logicznych do pinów.

Święta Księga C++

Tom III: Zarządzanie Zmianą (Gracz.h)

1. Dziedziczenie (extends)

```
class Gracz : public Postac { ... }
```

Słowo **public** po dwukropku oznacza, że Gracz otrzymuje cały publiczny i chroniony (protected) inwentarz z klasy **Postac**. Nie musimy drugi raz pisać pancerza, HP czy funkcji otrzymywania obrażeń. Gracz JEST Postacią, ma tylko dobudowane nowe moduły.

2. Wektory (Elastyczna Tablica)

```
#include <vector>
std::vector<Karta> reka;
```

Zwykła tablica w C++ **Karta reka[5];** jest sztywna — ma zawsze rozmiar 5. Wektor to inteligentna tablica dynamiczna. Kiedy używamy **reka.push_back(k)**, wektor sam rezerwuje nowe miejsce w pamięci komputera i dokleja kartę na koniec listy, a kiedy karta jest usuwana, sam tę pamięć zwalnia.

3. Przekazywanie parametrów wyżej (Super-konstruktor)

```
Gracz(std::string n, int startHp, int startPA) : Postac(n, startHp) { ... }
```

Ponieważ Gracz dziedziczy po Postaci, przy jego tworzeniu musimy też uruchomić maszynę w klasie Postać. Ten fragment **: Postac(n, startHp)** "podaje dalej" nazwę i HP do konstruktora bazowego, podczas gdy sam Gracz zajmuje się tylko przypisaniem swoich Punktów Akcji.

Święta Księga C++

Tom IV: Sztuczna Inteligencja (Boss.h)

1. Maszyna Stanów (State Machine)

```
void aktualizujFaze() {  
    if (faza == 1 && hp <= (maxHp / 2)) {  
        faza = 2;  
    }  
}
```

To najprostsza forma maszyny stanów. Zamiast liczyć ułamki przy każdym ataku, obiekt posiada własny stan w pamięci (Faza 1 lub 2). Przejście do nowego stanu jest jednokierunkowe (triggerowane spadkiem HP). To klasyczna pętla decyzyjna układu: odczytaj czujnik (HP), jeśli próg przekroczony – zmień stan logiczny.

2. Hermetyzacja Logiki

```
int wykonajAtak() {  
    aktualizujFaze();  
    if (faza == 1) return 15;  
    else return 30;  
}
```

Interfejs użytkownika (Qt) nie powinien wiedzieć, dlaczego Boss bije za 30 ani kiedy wchodzi w 2 fazę. Zgodnie z zasadą hermetyzacji, Qt wywołuje tylko `boss.wykonajAtak()` i dostaje w odpowiedzi suchą liczbę. Cała odpowiedzialność za proces myślowy jest bezpiecznie zamknięta (zhermetyzowana) wewnątrz klasy **Boss**.